Libraries and Dependencies

Libraries and external services Trading Strategy Lab actually uses. Each entry was checked against `backend/pyproject.toml`, `frontend/package.json`, and imports under `backend/app` and `frontend/src`.

Application code was not changed to produce this document.

Analysis date: 2026-10-08.

1. Overview

The application is a Python API plus a Next.js UI. Crew processes are the same Python package, started as modules rather than as separate services.

Surface	Manifest	Lockfile
Backend	`backend/pyproject.toml`	`backend/uv.lock`
Frontend	`frontend/package.json`	`frontend/package-lock.json`
Reference tree	`vendor/TradingAgents`	Not installed as a dependency of this app

Backend packages require Python `>=3.12`. The frontend declares Next.js `15.5.27` and React `19.1.0`.

Criticality	Meaning here
Core	Imported on the API or UI path that always loads
Supporting	Imported on a path that runs only for that feature
Development-only	Tests or lint
Unused declaration	Listed in a manifest or `.env.example` and not imported

`vendor/TradingAgents` declares its own dependencies. This application does not import that package. Those dependencies are not listed below.

2. Backend libraries

FastAPI

1. **Name:** FastAPI

2. **Version:** `>=0.142.2` (declared)

3. **Category:** API

4. **Purpose:** HTTP routes, request models, and OpenAPI.

5. **Why it is used:** The API is a FastAPI app.

6. **Where it is used:** `backend/app/main.py`, `backend/app/api/routes.py`, `journal.py`, `audit.py`, `live.py`, `common.py`.

7. **Main functionality:** `FastAPI`, `APIRouter`, `Depends`, `HTTPException`.

8. **Important configuration:** `BACKEND_PORT` (default 8841), `CORS_ORIGINS`.

9. **Criticality:** Core.

10. **External service:** None.

11. **Compatibility:** Pydantic v2.

Uvicorn

1. **Name:** Uvicorn

2. **Version:** `>=0.54.0`

3. **Category:** API

4. **Purpose:** ASGI server.

5. **Why it is used:** The root README starts `uvicorn app.main:app`.

6. **Where it is used:** Process entry. `backend/app/main.py` does not call `uvicorn.run`. The console script `trading-strategy-lab-backend` points at `trading_strategy_lab_backend:main`, which only prints a hello string.

7. **Criticality:** Core for serving the API.

8. **External service:** None.

Pydantic and pydantic-settings

1. **Name:** pydantic, pydantic-settings

2. **Version:** `pydantic>=2.13.5`, `pydantic-settings>=2.15.0`

3. **Category:** Validation and configuration

4. **Purpose:** Request bodies and `Settings`.

5. **Where it is used:** `backend/app/config.py`, API request models.

6. **Important configuration:** Environment variables documented in `.env.example`. `Settings` ignores unknown keys (`extra="ignore"`), which is why unused Langfuse keys do not crash startup.

7. **Criticality:** Core.

python-dotenv

1. **Name:** python-dotenv

2. **Version:** `>=1.2.4`

3. **Category:** Configuration

4. **Purpose:** Load `.env` from the working directory.

5. **Where it is used:** `load_dotenv()` in `backend/app/main.py`, `app/live/__main__.py`, `app/agents/trade_crew/__main__.py`. `dotenv_values` in `backend/app/brokers/credentials.py` reads `BROKER_*` keys.

6. **Criticality:** Core when configuration comes from `.env`.

SQLAlchemy

1. **Name:** SQLAlchemy
2. **Version:** >=2.0
3. **Category:** Database
4. **Purpose:** ORM, sessions, and metadata for all three databases.
5. **Where it is used:** backend/app/db/, backend/app/market/models.py.
6. **Criticality:** Core.
7. **External service:** SQLite file or PostgreSQL.

Alembic

1. **Name:** Alembic
2. **Version:** >=1.20.0
3. **Category:** Database
4. **Purpose:** Research and live migrations.
5. **Where it is used:** backend/alembic, backend/alembic_live, backend/alembic.ini. Tests apply the migrations.
6. **Criticality:** Core for a migrated database. The market database does not use it.
7. **Note:** The market database calls create_all instead.

3. AI / LLM libraries

LangGraph

1. **Name:** langgraph
2. **Version:** >=1.2.14
3. **Category:** Agent orchestration
4. **Purpose:** State graphs for the Review Crew and the Trade Crew.
5. **Where it is used:** backend/app/agents/review_crew/graph.py, backend/app/agents/trade_crew/graph.py (StateGraph, START, END).
6. **Criticality:** Core for review and for each crew tick.
7. **External service:** None. The graph itself is in-process.

LangChain core, OpenAI, and Anthropic

1. **Name:** langchain-core, langchain-openai, langchain-anthropic
2. **Version:** langchain-core>=1.6.7, langchain-openai>=1.6.7, langchain-anthropic>=1.0
3. **Category:** LLM clients

4. **Purpose:** Chat completions for extract, improvement rationale, opinions, lessons, and the optional explain note.

5. **Where it is used:** `backend/app/llm.py` constructs `ChatOpenAI` or `ChatAnthropic`. `backend/app/extract/extractor.py` imports `HumanMessage` and `SystemMessage`.

6. **Important configuration:** `LLM_PROVIDER`, `OPENAI_API_KEY`, `OPENAI_MODEL`, `ANTHROPIC_API_KEY`, `ANTHROPIC_MODEL`, `LLM_OPINION_MODEL`, `LLM_OPINION_TIMEOUT_S`.

7. **Criticality:** Supporting. The demo fixture and a rules-only crew tick run without a key.

8. **External service:** OpenAI or Anthropic, one of them, matching `LLM_PROVIDER`.

openai (declared package)

1. **Name:** openai

2. **Version:** `>=3.26.0`

3. **Category:** LLM SDK

4. **Purpose:** Declared directly. Application code does not `import openai`. `ChatOpenAI` uses this SDK.

5. **Criticality:** Supporting, transitive in practice, direct in the manifest.

6. **External service:** OpenAI, only when `LLM_PROVIDER=openai` and a key is set.

4. Ingest libraries

HTTPX

1. **Name:** httpx

2. **Version:** `>=0.28.1`

3. **Category:** HTTP client

4. **Purpose:** Fetch HTML and Instagram pages. Send Telegram alerts.

5. **Where it is used:** `backend/app/ingest/doc.py`, `instagram.py`, `backend/app/alerts.py`.

6. **Criticality:** Supporting.

7. **External service:** The URL the user submitted, and `api.telegram.org` when alerts are configured.

Beautiful Soup and lxml

1. **Name:** beautifulsoup4, lxml

2. **Version:** `beautifulsoup4>=4.15.0`, `lxml>=6.1.3`

3. **Category:** HTML

4. **Purpose:** Strip scripts and layout from a document URL.

5. **Where it is used:** `backend/app/ingest/doc.py` (`BeautifulSoup(html, "lxml")`). `lxml` is not imported by name. It is the parser backend.

6. **Criticality:** Supporting. PDF responses are rejected before parsing.

youtube-transcript-api

1. **Name:** youtube-transcript-api

2. **Version:** >=1.2.4

3. **Category:** Ingest

4. **Purpose:** Fetch a YouTube transcript.

5. **Where it is used:** backend/app/ingest/youtube.py.

6. **Criticality:** Supporting.

7. **External service:** YouTube.

PyYAML

1. **Name:** pyyaml

2. **Version:** >=6.0.3

3. **Category:** Documents

4. **Purpose:** Strategy frontmatter, holiday list, event files.

5. **Where it is used:** backend/app/strategies/schema.py, backend/app/market/calendar.py, backend/app/market/hours.py.

6. **Criticality:** Core for any strategy with frontmatter.

5. Security

cryptography

1. **Name:** cryptography

2. **Version:** >=44

3. **Category:** Secrets

4. **Purpose:** Fernet encryption for broker_credentials.ciphertext.

5. **Where it is used:** backend/app/brokers/credentials.py.

6. **Important configuration:** BROKER_CREDENTIALS_KEY. Without a valid key, save returns 400 and stored values are ignored.

7. **Criticality:** Supporting. Required before credentials can be stored.

8. **External service:** None.

API token checks use the standard library hashlib (SHA-256). There is no Auth0, PyJWT, or session library.

SMTP alerts use the standard library smtplib.

6. Observability

No tracing or metrics library is imported.

`LANGFUSE_SECRET_KEY`, `LANGFUSE_PUBLIC_KEY`, and `LANGFUSE_BASE_URL` appear in `.env.example` only. They are not fields on `Settings` and have no client. Treat them as unused.

Audit chains, heartbeats, and alerts are application code (`backend/app/audit/`, `crew_heartbeats`, `backend/app/alerts.py`).

7. Frontend libraries

Next.js

1. **Name:** next
2. **Version:** `15.5.27`
3. **Category:** UI
4. **Purpose:** App router and dev server.
5. **Where it is used:** `frontend/src/app/`.
6. **Scripts:** `next dev --turbopack`, `next build --turbopack`.
7. **Criticality:** Core for the UI.
8. **Configuration:** `NEXT_PUBLIC_API_URL`.

React and React DOM

1. **Name:** react, react-dom
2. **Version:** `19.1.0`
3. **Category:** UI
4. **Purpose:** Page components. No extra component library.
5. **Criticality:** Core.

Tailwind CSS

1. **Name:** tailwindcss, `@tailwindcss/postcss`
2. **Version:** `^4`
3. **Category:** UI
4. **Purpose:** Utility classes on the pages and components.
5. **Criticality:** Core for the current layout.

The UI does not depend on an HTTP client library. `frontend/src/lib/api.ts` uses `fetch`.

8. Testing and lint

Package	Where	Criticality
pytest >=9.1.1	backend dev group. uv run pytest	Development-only
eslint ^9, eslint-config-next 15.5.27	frontend npm run lint	Development-only
typescript ^5, @types/react, @types/node	Frontend compile	Development-only

Tests do not call a live LLM or a network broker. They migrate temporary SQLite files.

9. External services

Service	When it is contacted	Required to boot
OpenAI	LLM_PROVIDER=openai and a key, on extract, rationale, opinion, lesson, or explain	No
Anthropic	LLM_PROVIDER=anthropic and a key, same call sites	No
YouTube	Ingest source type youtube	No
Instagram	Ingest source type instagram	No
Document URL	Ingest source type document	No
Telegram	Both ALERT_TELEGRAM_BOT_TOKEN and ALERT_TELEGRAM_CHAT_ID set	No
SMTP server	ALERT_SMTP_HOST set	No
NSE hosts	Only the import scripts, which download or read bhavcopy files the operator points them at	No
Langfuse	Never, with the current code	No

Paper trading on the replay feed needs none of these.

10. Potentially unused or reference-only

Item	Evidence
Langfuse environment keys	.env.example only
openai import	No import openai under backend/. The package is still required by langchain-openai
trading_strategy_lab_backend console script	backend/src/trading_strategy_lab_backend/__init__.py prints a hello string and does not start Uvicorn
vendor/TradingAgents	Not on sys.path for the app. See vendor/INTEGRATION.md
psycopg	Mentioned for Postgres URLs. Not in pyproject.toml

11. Dependency summary

Package	Criticality	Direct import in backend/app or frontend/src
fastapi	Core	Yes

Package	Criticality	Direct import in `backend/app` or `frontend/src`
uvicorn	Core at runtime	Started by the operator, not imported in `app`
pydantic, pydantic-settings	Core	Yes
python-dotenv	Core	Yes
sqlalchemy	Core	Yes
alembic	Core for migrations	Yes, in migration env modules
langgraph	Core for crews	Yes
langchain-core	Supporting	Yes
langchain-openai	Supporting	Yes, inside `chat_model`
langchain-anthropic	Supporting	Yes, inside `chat_model`
openai	Supporting	No direct import
httpx	Supporting	Yes
beautifulsoup4	Supporting	Yes
lxml	Supporting	Parser name only
youtube-transcript-api	Supporting	Yes
pyyaml	Core for frontmatter	Yes
cryptography	Supporting	Yes
pytest	Development-only	Tests
next, react, react-dom, tailwindcss	Core for the UI	Yes

Summary

The running system is FastAPI, SQLAlchemy, Alembic, LangGraph, and a Next.js UI. LLM calls go through LangChain to either OpenAI or Anthropic. HTML ingest uses HTTPX, Beautiful Soup, and lxml. YouTube ingest uses `youtube-transcript-api`. Broker secrets use Fernet. There is no vector database, no Auth0, and no Langfuse client in this repository.